explain terraform directory structure

A **Terraform directory structure** is how you organize Terraform configuration files so infrastructure code is **modular, maintainable, reusable, and environment-specific**. This becomes very important in **real DevOps projects** where multiple environments (dev, stage, prod) and many resources exist.

1️⃣ Basic Terraform Directory Structure (Small Project)

```
terraform-project/
│
├── main.tf
├── variables.tf
```

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

Explanation

1. main.tf

- Main infrastructure definition
- Calls modules and resources

Example

```hcl
resource "aws_instance" "web" {
  ami           = var.ami
  instance_type = var.instance_type
}
```

---

2. variables.tf

Defines input variables.

```hcl
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
}
```

---

3. outputs.tf

Outputs useful information after apply.

```hcl
output "instance_ip" {
  value = aws_instance.web.public_ip
}
```

---

4. providers.tf

Defines provider configuration.

```hcl
provider "aws" {
  region = "ap-south-1"
}
```

---

5. terraform.tfvars

Stores actual variable values.

```hcl
instance_type = "t3.micro"
ami           = "ami-0abcdef"
```

---

6. backend.tf

Defines remote state backend.

Example (S3 backend)

```hcl
terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "ap-south-1"
  }
}
```

2️⃣ Production Terraform Directory Structure (Best Practice)

In real companies we use **modules + environment separation**.

```
terraform-infra/
│
├── modules/
│   ├── vpc/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   │
│   ├── ecs/
│   ├── rds/
│   └── alb/
│
├── environments/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── backend.tf
│   │   ├── terraform.tfvars
│   │
│   ├── stage/
│   │
│   └── prod/
│
└── global/
    ├── providers.tf
    └── variables.tf
```

3️⃣ Modules Folder

Modules contain **reusable infrastructure components**.

Example

```
modules/vpc/
   main.tf
   variables.tf
   outputs.tf
```

Example module code:

```
modules/vpc/main.tf
```

```hcl
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr
}
```

4️⃣ Environment Folder

Each environment has **its own state and configuration**.

```
environments/prod/
```

Example

```hcl
module "vpc" {
  source   = "../../modules/vpc"
  vpc_cidr = "10.0.0.0/16"
}
```

Example tfvars

```
vpc_cidr = "10.0.0.0/16"
```

This allows:

- dev → smaller infrastructure
- prod → bigger infrastructure

5️⃣ Terraform Files Terraform Automatically Loads

Terraform automatically loads files ending with:

```
*.tf
*.tfvars
```

So these all combine:

```
main.tf
network.tf
compute.tf
database.tf
```

Terraform treats them as **one configuration**.

6️⃣ Real DevOps Terraform Structure (Very Common in Companies)

```
terraform/
│
├── modules/
│   ├── vpc
│   ├── ecs-service
│   ├── alb
│   ├── rds
│
├── env/
│   ├── dev
│   ├── qa
│   ├── prod
│
├── scripts/
│   ├── init.sh
│   └── deploy.sh
│
└── global/
    ├── backend.tf
    └── provider.tf
```

7️⃣ Example: ECS Infrastructure Layout (Real Scenario)

```
terraform/
│
├── modules/
│   ├── vpc
│   ├── ecs-cluster
│   ├── ecs-service
│   ├── alb
│   └── iam
│
├── env/
│   ├── dev
│   │   ├── main.tf
│   │   └── terraform.tfvars
│   │
│   └── prod
│       ├── main.tf
│       └── terraform.tfvars
```

8️⃣ Why This Structure is Important (Interview Answer)

Benefits:

1️⃣ **Reusability**
Modules allow reuse across environments.

2️⃣ **Environment Isolation**
Dev, Stage, Prod separated.

3️⃣ **State Management**
Different Terraform states.

4️⃣ **Team Collaboration**

5️⃣ **Scalable Infrastructure**

9️⃣ Interview Answer (Short Version)

If interviewer asks:

"How do you structure Terraform in projects?"

Answer:

> We follow a modular Terraform structure where reusable infrastructure components like VPC, ECS, ALB, and RDS are defined inside a `modules` directory.
> Environment-specific configurations such as dev, staging, and production are maintained in separate folders.
> Each environment references the modules and uses its own `terraform.tfvars` and remote backend configuration to maintain separate state files.

✅ If you want, I can also explain **a complete Terraform project used in companies (ECS + ALB + RDS + VPC)** — this is a **very common DevOps interview question.**

```
terraform-infra/
│
├── modules/
│   ├── vpc/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   │
│   ├── ecs/
│
├── environments/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── backend.tf
│   │   ├── terraform.tfvars
│   │
│   ├── stage/
│
└── global/
    ├── providers.tf
    ├── variables.tf
    └── provider.tf
```

in this explain if i want to use separate backend for storing state files in sperate s3 buckets for dev & stage . how can i do it ? is main.tf file necessary in the global ? difference between variables.tf & terraform.tfvars & precedence .

Good question — this is exactly how Terraform is used in **real production repositories**. Let's break your questions one by one.

1️⃣ Separate S3 Backend for Dev & Stage

Since your environments are separated:

```
terraform-infra/
│
├── environments/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── backend.tf
│   │   └── terraform.tfvars
│   │
│   ├── stage/
│   │   ├── main.tf
│   │   ├── backend.tf
│   │   └── terraform.tfvars
```

Each environment can have **its own backend configuration**.

Dev backend example

`environments/dev/backend.tf`

```hcl
terraform {
  backend "s3" {
    bucket         = "terraform-dev-state-bucket"
    key            = "dev/terraform.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "terraform-lock-dev"
    encrypt        = true
  }
}
```

Stage backend example

`environments/stage/backend.tf`

```hcl
terraform {
  backend "s3" {
    bucket         = "terraform-stage-state-bucket"
    key            = "stage/terraform.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "terraform-lock-stage"
    encrypt        = true
  }
}
```

Deployment command

You run Terraform **from the environment directory**.

Example:

```Bash
cd environments/dev

terraform init
terraform plan
terraform apply
```

Stage:

```Bash
cd environments/stage

terraform init
terraform plan
terraform apply
```